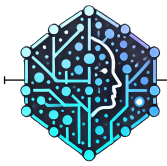


LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. AI APPLICATIONS IN PRACTICE

9 - Code Versioning and Git

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

Code Versioning

Version control systems (VCS) are tools that help track changes to files over time, allowing you to recall specific versions later and collaborate efficiently.

Key benefits:

- Track changes to your code
- Collaborate with others without conflicts
- Revert to previous versions if needed
- Document why changes were made
- Work on multiple features simultaneously

Why use version control?

Consider these scenarios:

- You want to try a new approach without breaking working code
- Multiple team members need to modify the same files
- You need to track which changes introduced a bug
- You want to maintain multiple versions (e.g., stable and development)
- You need to document the evolution of your codebase

Why use version control?

Without version control, you might resort to:

- Copying files manually (project_v1, project_v2, etc.)
- Using comments to track changes
- Sharing files via email or cloud storage
- Manually merging everyone's changes

Types of version control systems

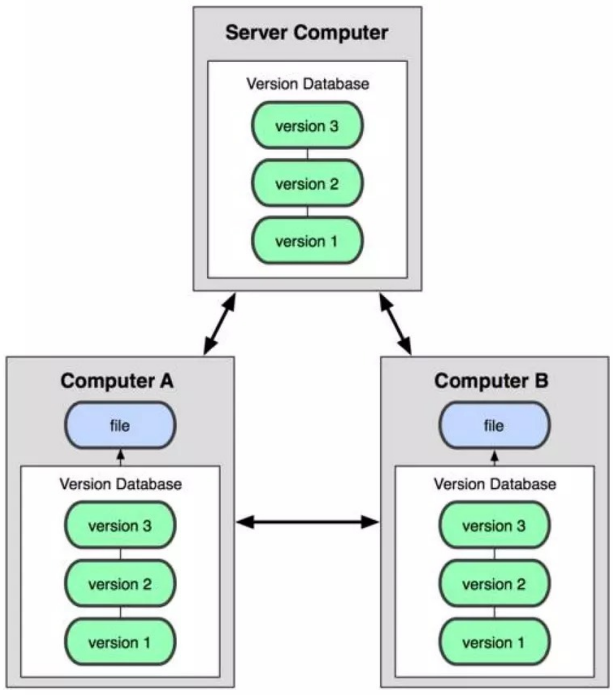
Version control systems have evolved over time:

- **Local VCS:** simple database on your local machine that tracks changes (e.g., RCS)
- **Centralized VCS:** single server storing all versioned files (e.g., SVN, Perforce)
- **Distributed VCS:** every user has a full copy of the repository (e.g., Git, Mercurial)

Most modern development uses distributed VCS, with Git being the most popular.

Distributed VCS

Each computer that has a copy of the repository, shares the whole version database.



Introduction to Git

Git is a distributed version control system created by Linus Torvalds in 2005 for Linux kernel development.

Key features:

- Distributed architecture (no single point of failure)
- Speed and efficiency (optimized for performance)
- Data integrity (uses SHA-1 hashing)
- Branching and merging (lightweight and powerful)
- Non-linear development (supports parallel workflows)

Introduction to Git

Git is the program and the whole VCS system including formats and interactions.

Github, GitLab, Bitbucket, etc. are servers that can host git repositories with its standard protocol, using HTTPS and SSH to communicate.

They allows to use a web UI to interact with the repository without having to take it locally.

Git installation

Installing Git is simple across various platforms:

Windows

- Download and install from https://git-scm.com/download/win
- Use Windows Subsystem for Linux (WSL)

macOS

- Install via Homebrew: brew install git
- Use the installer from https://git-scm.com/download/mac
- Comes pre-installed on newer versions of macOS

Linux

- Ubuntu/Debian: sudo apt-get install git
- Fedora: sudo dnf install git
- Arch Linux: sudo pacman -S git

Git first steps

After installation, configure your identity:

```
# Set your username and email
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set default text editor
git config --global core.editor "code --wait"  # For VS Code

# View your configuration
git config --list
```

These settings are stored in ~/.gitconfig (or C:\Users\<username>\.gitconfig on Windows).

Basic Git Concepts

Git considers its data as a series of **snapshots**.

Every time you edit a file, the status of your project in Git will be updated. Git basically makes an image of all the files at that time, saving a reference to the **snapshot**.

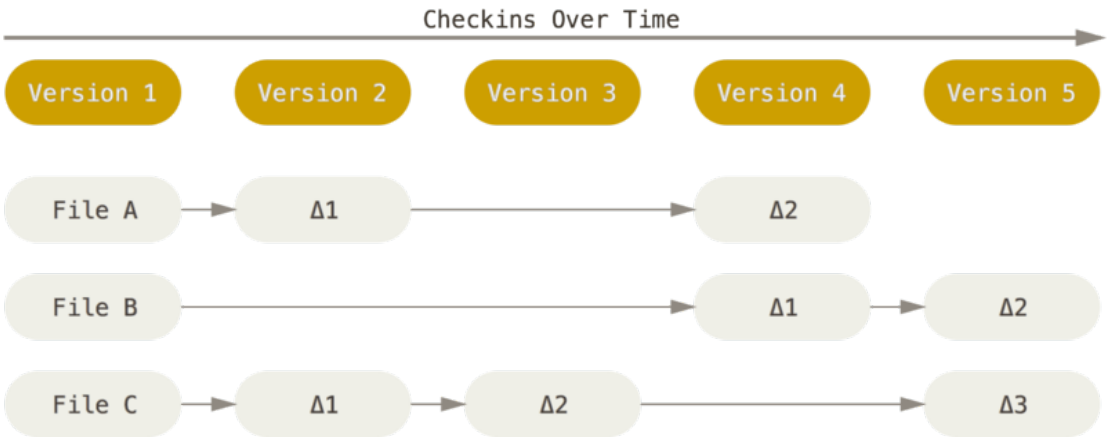
To be efficient, if some files have not changed, Git does not save them, but creates simply **a link** to the previous file already saved.

PRO: speed in reassembling the story in the face of an occupation of space not optimized

CONS: greater occupation of space compared to other solutions

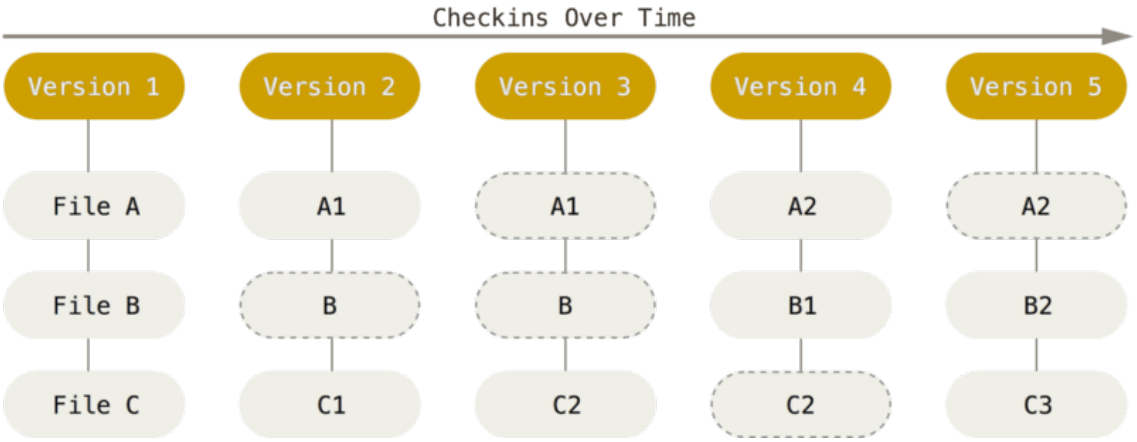
Basic Git Concepts

Other types of VCS usually consider a scheme that records deltas of files:



Basic Git Concepts

Git, instead, consider snapshots and links unmodified versions of files.



Basic Git Concepts

Almost every operation on the repository are **local**. They do not need to sync to other repositories until needed.

There is also the concept of **integrity,** each snapshot has a *SHA-1 hash* associated with, and they are used to link subsequent snapshots.

Typically the only action is to **add** data, that is every change is recorded incrementally. You do not usually rewind back and delete previous changes.

Basic Git Concepts

Understanding Git requires knowledge of a few core concepts:

- **Repository**: The entire collection of files and their history
- **Working Directory**: The files you are currently editing
- **Staging Area (Index)**: The intermediate area where changes are prepared
- **Commit**: A snapshot of your changes at a specific point in time
- **Branch**: A lightweight, movable pointer to a commit
- **Remote**: A remote repository (usually on a server)

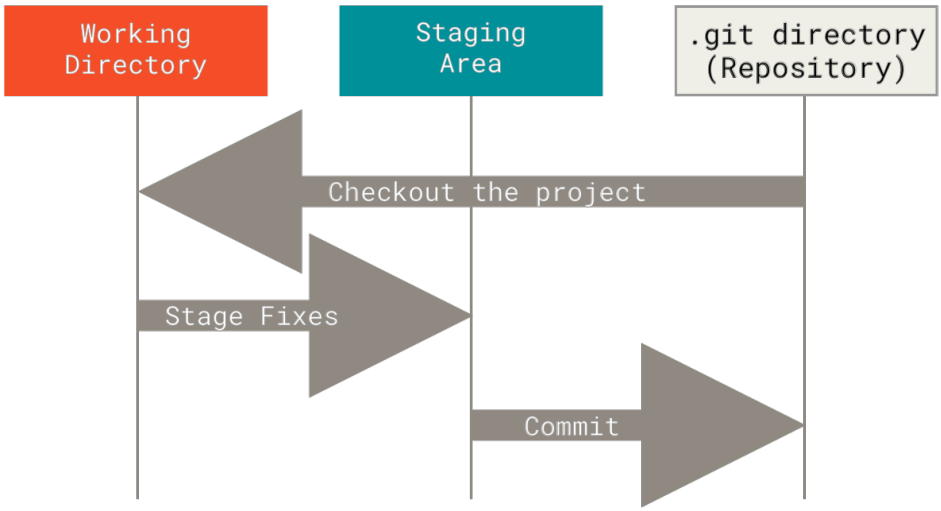
Three States

Git has three main states that your files can reside in: *modified*, *staged*, and *committed*:

- **Modified** means that you have changed the file but have not *committed* it to your database yet.
- **Staged** means that you have marked a modified file in its current version to go into your next commit snapshot.
- **Committed** means that the data is safely stored in your local database.

Three States

- The *working tree* is a single checkout of one version of the project.
- The *staging area* is a file, generally contained in your Git directory, that stores information about what will go into your next commit.
- The *Git directory* is where Git stores the metadata and object database for your project



Git Workflow Overview

A typical Git workflow follows these steps:

1. **Initialize** a repository or clone an existing one
2. **Create/modify** files in your working directory
3. **Stage** changes for commit (add to the index)
4. **Commit** changes (create a snapshot)
5. **Push** changes to a remote repository (if needed)
6. **Pull** changes from a remote repository (if needed)

Creating a Git Repository

You can start a Git repository in two ways:

- Initialize a new repository:

```
# Navigate to your project directory
cd my-project

# Initialize Git
git init
```

- Clone an existing repository:

```
# Clone from a remote source
git clone https://github.com/username/repository.git

# Clone to a specific directory
git clone https://github.com/username/repository.git my-folder
```


The .git directory

When you initialize a repository, Git creates a .git directory that contains:

- objects/: The object database (blobs, trees, commits)
- refs/: References to commits (branches, tags)
- HEAD: Points to the current branch
- config: Repository-specific configuration
- index: Staging area information

You generally don't need to interact with these files directly, but knowing they exist helps understand Git's structure.

Tracking Changes

The basic cycle of tracking changes in Git:

```
# Check the status of your files
git status

# Add files to the staging area
git add filename          # Add specific file
git add .                 # Add all changed files
git add -p                # Interactively add parts of files

# Commit the changes
git commit -m "Description of changes"

# View commit history
git log
```


Understanding Git Status

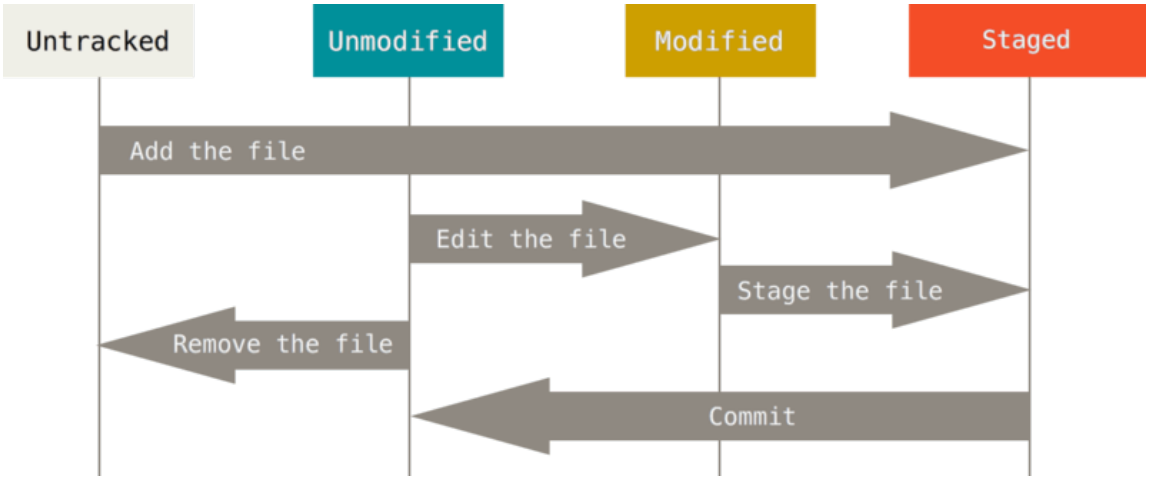
The ***git status*** command shows:

- Untracked files: new files Git doesn't know about
- Modified files: tracked files that have been changed
- Staged files: files ready to be committed
- Current branch: which branch you're working on

Running ***git status*** frequently helps you maintain awareness of your repository's state.

Recording changes

While changing the current files, they can be in four different states:



Git add and the staging area

The staging area (or index) is a key Git concept that lets you prepare what goes into your next commit.

The staging area allows you to:

- Commit only a subset of changes
- Review changes before committing
- Group related changes into separate commits

```
# Stage all modified and new files
git add .

# Stage specific files
git add file1.py file2.py

# Stage parts of files interactively
git add -p
```

Git commit

A commit creates a snapshot of your staged changes

```
# Create a commit with a message
git commit -m "Implement user authentication"

# Add all tracked files and commit in one step
git commit -am "Fix typo in readme"

# Amend the previous commit
git commit --amend
```

Each commit has:

- A unique identifier (SHA-1 hash)
- Author information
- Timestamp
- Commit message
- Pointer to the previous commit(s)

Commit messages

Good commit messages are essential for a maintainable codebase:

Structure:

Short summary line (< 50 characters)

Blank line

Detailed explanation if necessary

Best practices:

Use the imperative mood ("Add feature" not "Added feature")

Explain what and why, not how

Keep it concise but informative

Reference issue numbers if applicable

Reference: https://cbea.ms/git-commit/

Commit messages

Example:

```
Add user authentication functionality

- Implement JWT-based authentication
- Add password hashing with bcrypt
- Create login and signup endpoints

Closes #45
```


Commit messages

https://xkcd.com/1296/

Viewing History

Git provides several ways to view your project's history:

```
# Basic commit history
git log

# Compact view
git log --oneline

# Visual graph of branches
git log --graph --oneline --all

# History for a specific file
git log -- path/to/file

# Show changes in commits
git log -p
```


Commit messages

The separation of subject from body pays off when browsing the log.

```
$ git log
commit 42e769bdf4894310333942ffc5a15151222a87be
Author: Kevin Flynn <kevin@flynnsarcade.com>
Date:   Fri Jan 01 00:00:00 1982 -0200

 Derezz the master control program

 MCP turned out to be evil and had become intent on world domination.
 This commit throws Tron's disc into MCP (causing its deresolution)
 and turns it back into a chess game.
```

```
$ git log --oneline
42e769 Derezz the master control program
```

Git Diff

The *git diff* command shows differences between states:

```
# Unstaged changes (working directory vs. staging area)
git diff

# Staged changes (staging area vs. last commit)
git diff --staged

# Differences between two commits
git diff commit1 commit2

# Differences in a specific file
git diff -- path/to/file
```


Undoing changes

Git provides several ways to undo changes:

- Unstaged changes:

```
git restore <file>            # Discard changes in working directory (Git 2.23+)
git checkout -- <file>        # Older syntax
```

- Staged changes:

```
git restore --staged <file>   # Unstage changes (Git 2.23+)
git reset HEAD <file>         # Older syntax
```

- Committed changes:

```
git revert <commit>           # Create a new commit that undoes changes
git reset --soft <commit>     # Move HEAD, keep changes staged
git reset --mixed <commit>    # Move HEAD, keep changes unstaged
git reset --hard <commit>     # Move HEAD, discard changes (DANGEROUS)
```

Undoing changes

Which one to use?

- Do you want to cancel a commit without rewriting the story?
  → *git revert*
- Do you want to rewrite the story keeping the changes?
  → *git reset --soft*
- Do you want to rewrite the history and report the changes in the working directory?
  → *git reset --mixed*
- Do you want to permanently delete commits and changes?
  → *git reset --hard* (only if you're sure!)

.gitignore

The *.gitignore* file specifies intentionally untracked files to ignore.

You can find templates for different languages at https://github.com/github/gitignore

```
# Ignore build directories
build/
dist/

# Ignore compiled files
*.o
*.pyc
__pycache__/

# Ignore environment files
.env
.venv/
venv/

# Ignore IDE-specific files
.idea/
.vscode/

# Ignore logs
*.log
```

Exercise

Exercise 1: Basic Git workflow

- Create a new repository
- Add some files
- Make changes and commit them
- View the commit history

Exercise 2: Reverting modifications

- Make changes and commit them
- View the commit history
- Cancel the commit without changing the history
- View the commit history
- Make some more changes and commit them
- View again the commit history

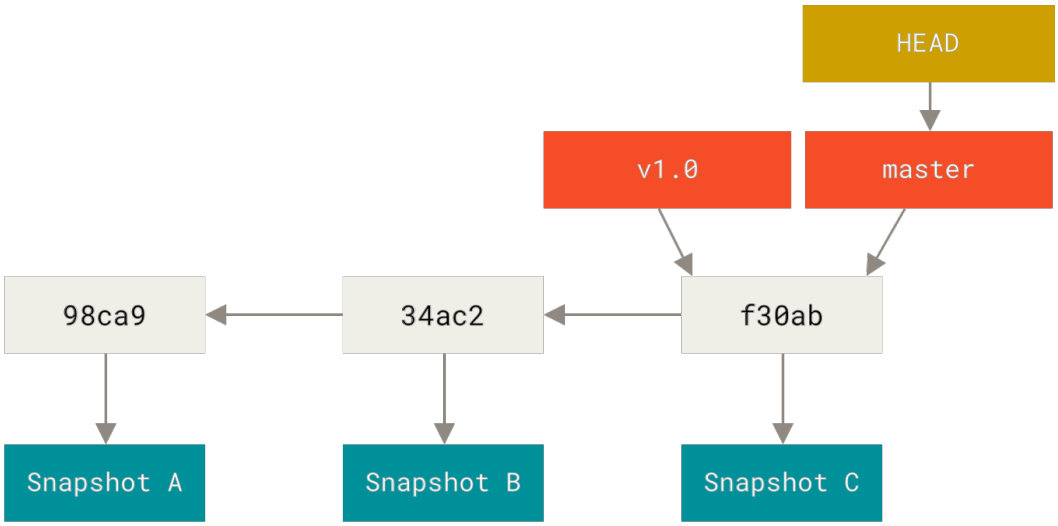
Branching

Branches allow parallel development tracks. They are lightweight pointers to commits, making them fast and easy to create.

```
# List branches
git branch

# Create a new branch
git branch feature-name

# Switch to a branch
git checkout feature-name     # Older syntax
git switch feature-name       # Git 2.23+

# Create and switch in one command
git checkout -b feature-name  # Older syntax
git switch -c feature-name    # Git 2.23+
```

Branching

Here there are two pointers, v1.0 and master.
The local repository in the working directory is at master (HEAD).

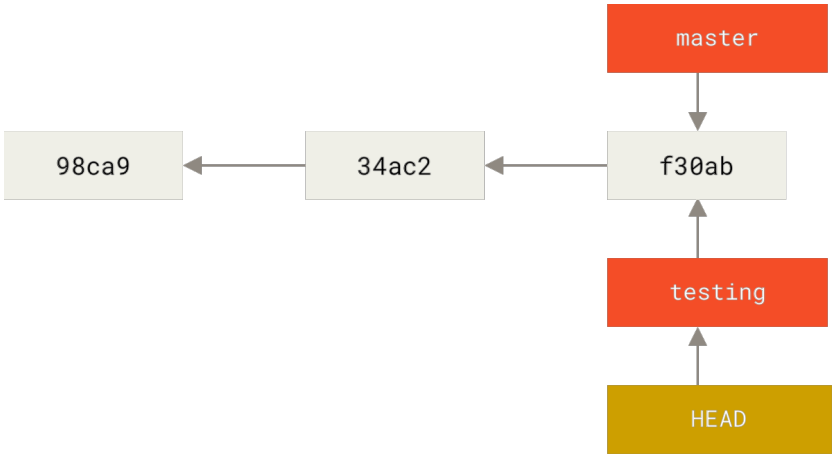


Branching

After executing *git branch testing*

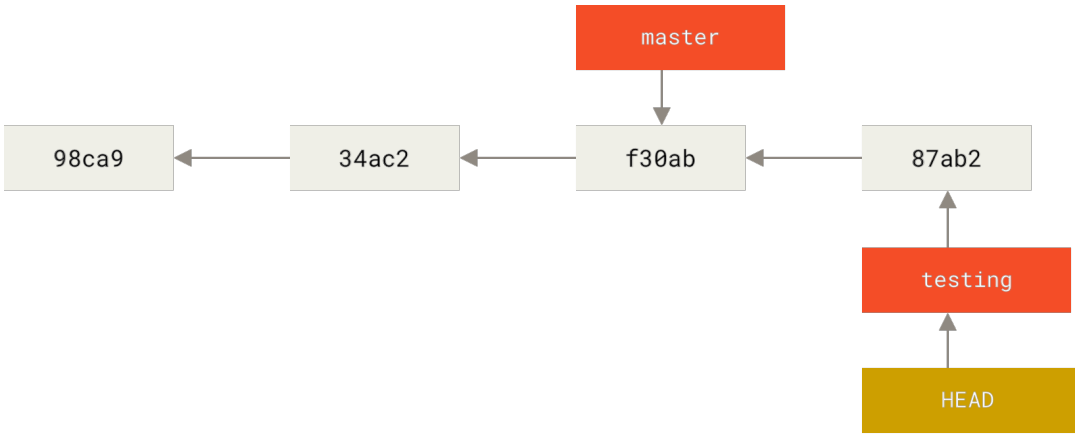
Branching

After executing *git checkout testing*



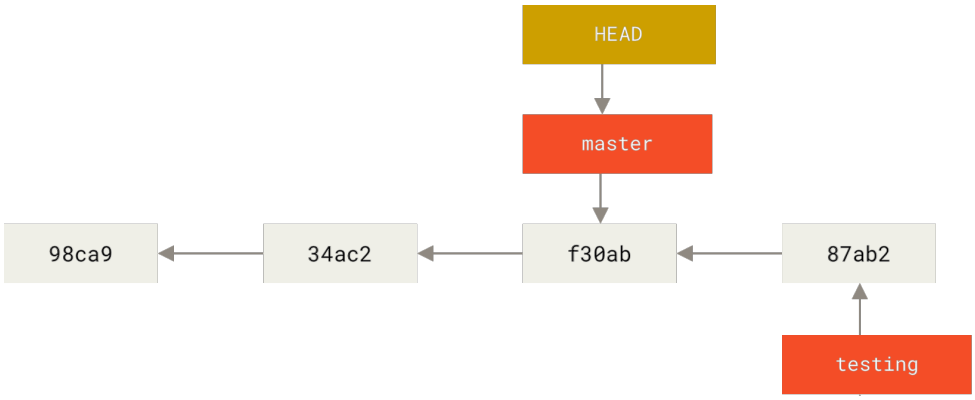
Branching

After making a commit *git commit -a -m 'Make a change'*



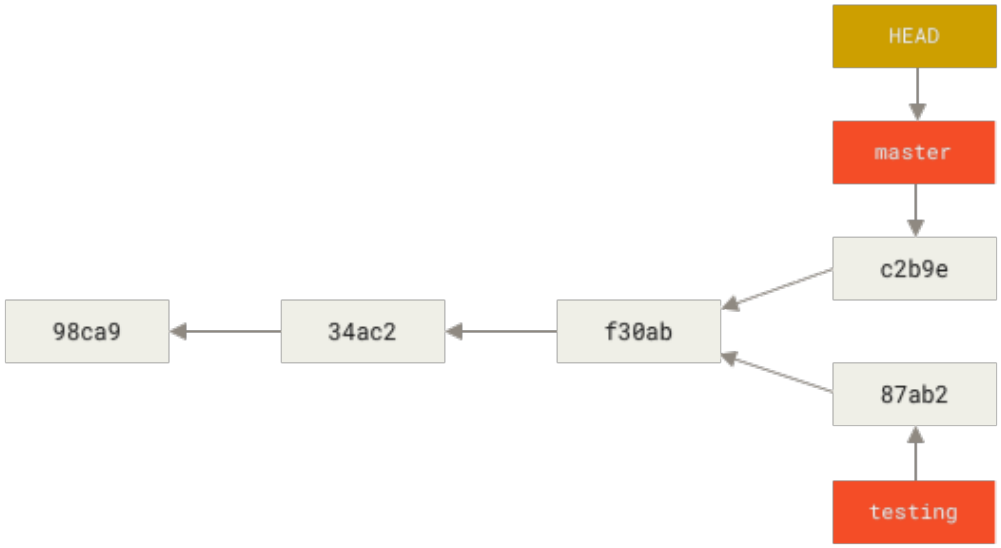
Branching

We can go back to master with *git checkout master*



Branching

If we commit something in the master branch
git commit -a -m 'Make other changes'



Merging

Merging combines changes from different branches.

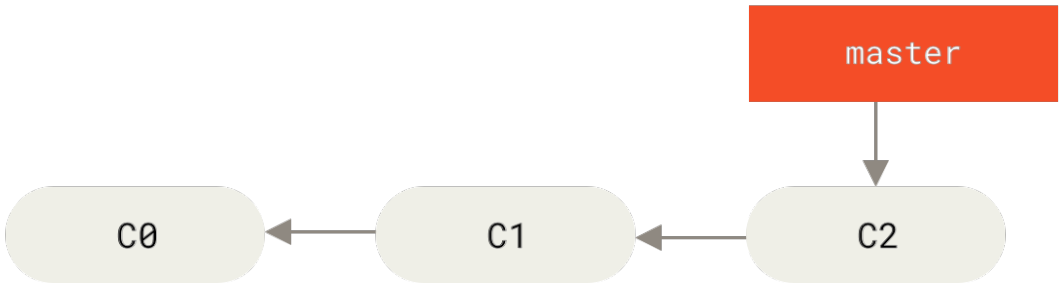
There are 3 types of merges:

- **Fast-forward**: when there are no new changes in the target branch
- **Recursive merge**: creates a merge commit combining both histories
- **Squash merge**: combines all changes into a single commit

```
# Switch to the target branch
git checkout main

# Merge another branch into the current branch
git merge feature-branch

# Create a merge commit
git merge --no-ff feature-branch
```


A possible workflow

Let's say you're working on your project and have a couple of commits already on the master branch.

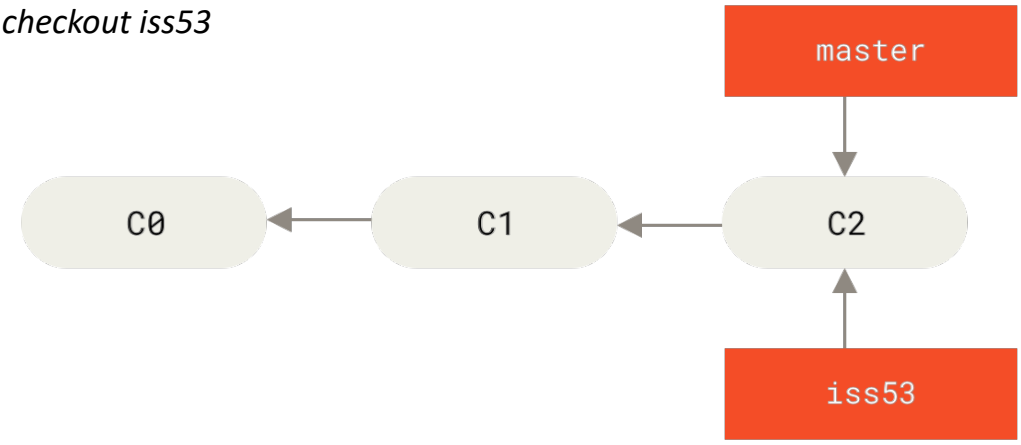


A possible workflow

You've decided that you're going to work on issue #53.

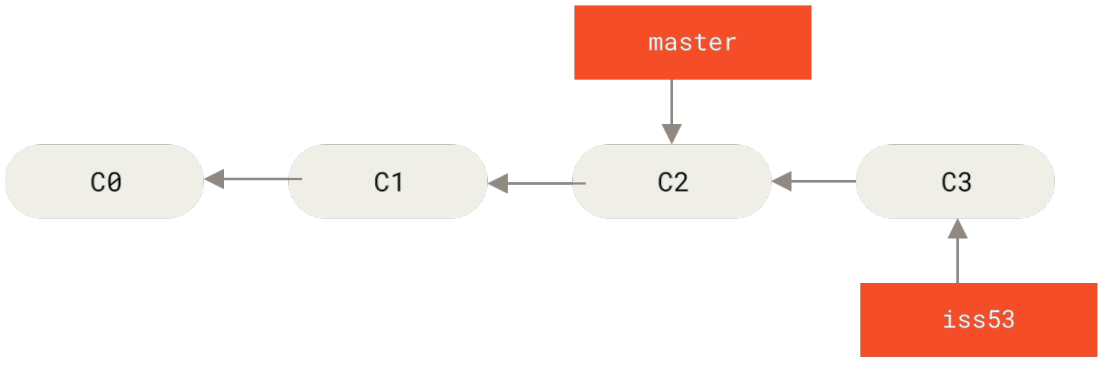
Let's create a new branch for it and switch to it:
git checkout -b iss53

The alternative two-step way is:
git branch iss53
git checkout iss53



A possible workflow

You work on your website and do a commit.
This moves the pointer of iss53 forward:
git commit -a -m 'Create new footer [issue 53]'



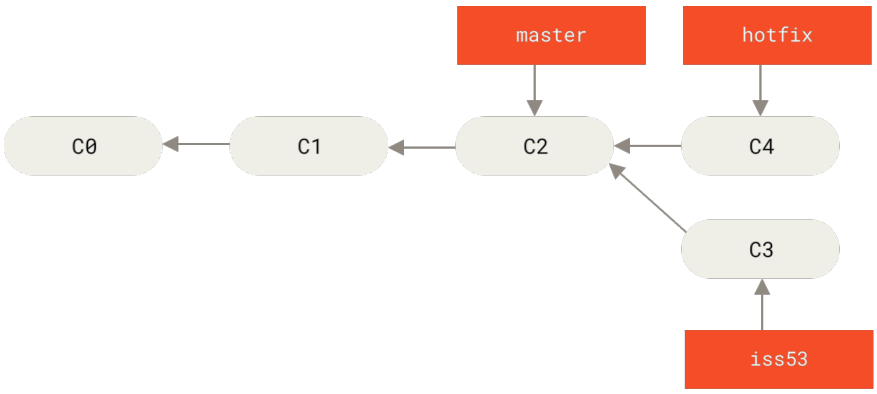
A possible workflow

At some point, let's say we need to fix a super important urgent bug before continuing to work on this issue. We create a new branch hotfix starting from master:

git checkout master

git checkout -b hotfix

…
git commit -a -m 'Fix broken email address'

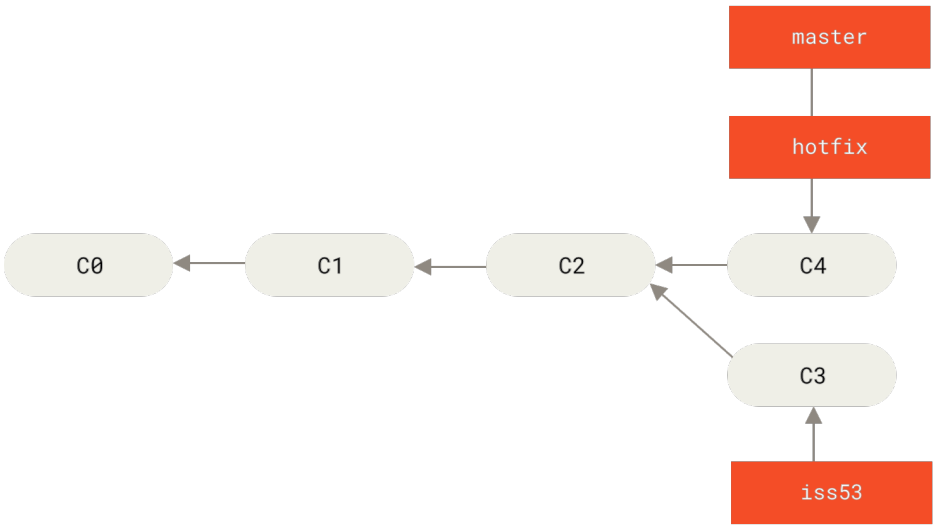


A possible workflow

After running tests, you decide that hotfix is fine, so we merge it into master (with a fast-forward merge):

git checkout master

git merge hotfix



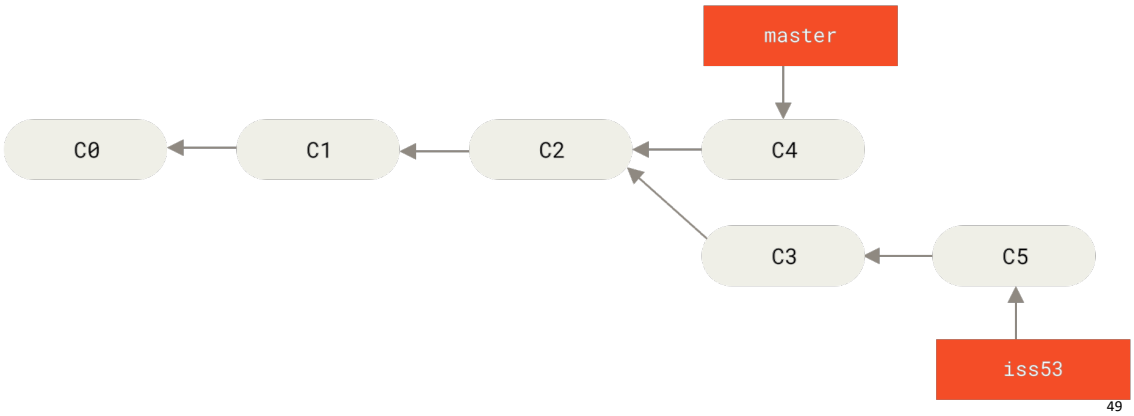
A possible workflow

Now you can switch back to your work-in-progress branch on issue #53 and continue working on it.

git checkout iss53

…

git commit -a -m 'Finish the new footer [issue 53]'

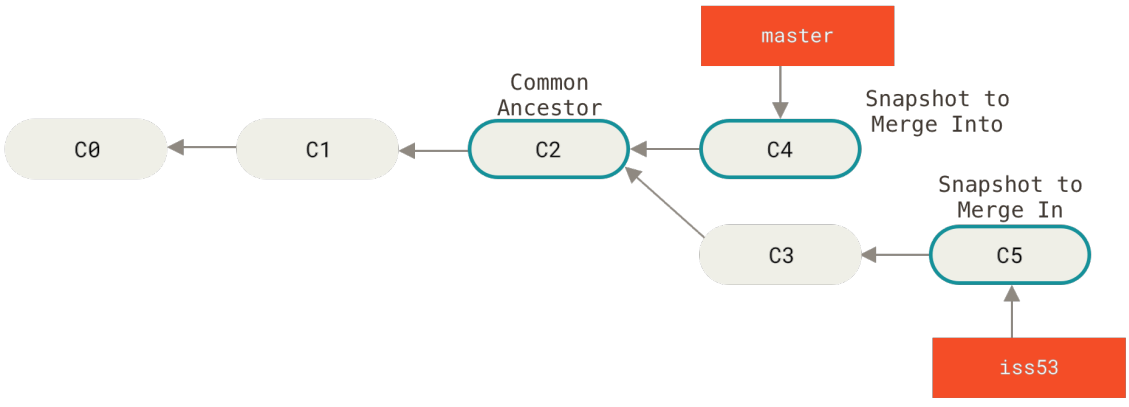


A possible workflow

Suppose work on issue #53 is complete. Now it's time to merge the new commits into master (using a recursive strategy):

git checkout master

git merge iss53

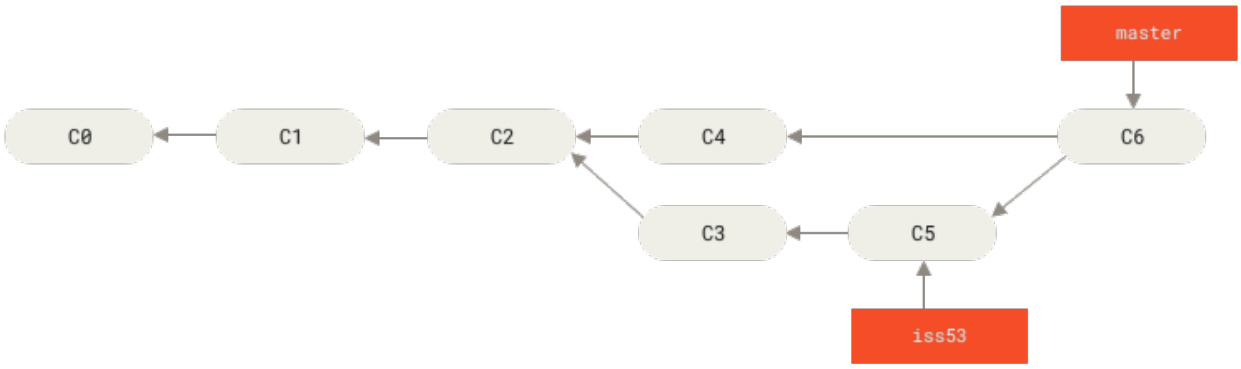


A possible workflow

Suppose work on issue #53 is complete. Now it's time to merge the new commits into master (using a recursive strategy):

git checkout master

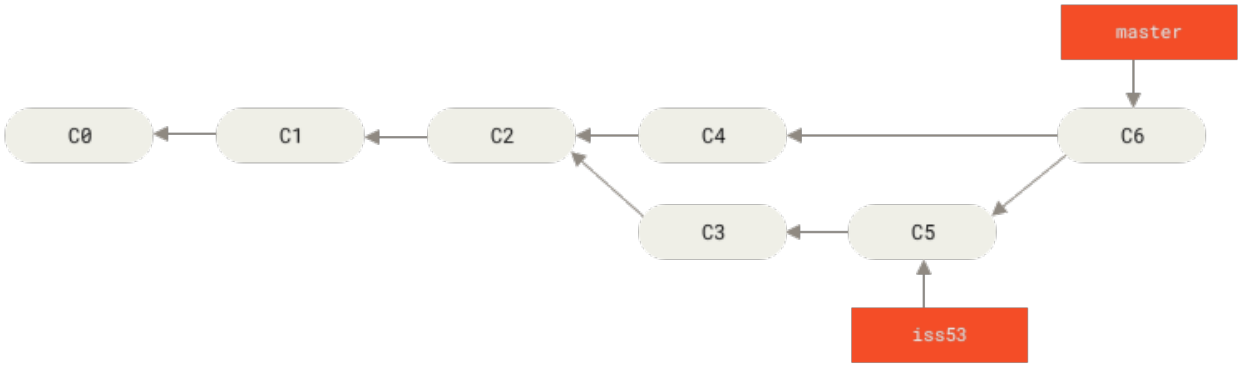
git merge iss53



A possible workflow

Finally we can delete the branch iss53

git branch -d iss53



Branching strategies

Branching is typically used according to some strategy. There are some typical ones, but the choice depends upon the organization (and your style).

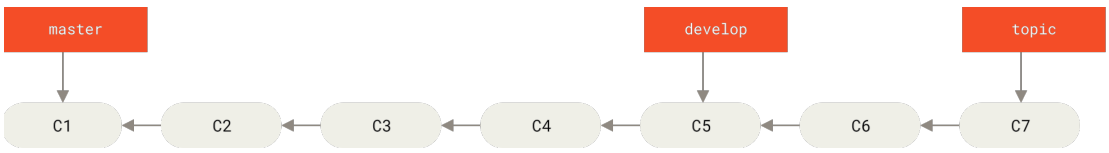
- **Git Flow:**

main (or *master*) for production releases
develop for development
*feature/** for new features
*release/** for preparing releases
*hotfix/** for emergency fixes

Branching strategies

- **Long-Running Branches:**

Master has entirely stable branch
Develop has current development version
Topics are short lived branches for issues or new features



Branching strategies

- **GitHub Flow:**

  main (or *master*) always deployable
  Feature branches for all changes
  Pull requests for review
  Merge to main and deploy

Resolving conflicts

Conflicts occur when Git can't automatically merge changes:

```
# Attempt to merge
git merge feature-branch
# (Conflict detected)

# Check status
git status

# Resolve conflicts in conflicted files
# Edit files to fix conflicts

# After resolving, mark as resolved
git add resolved-file.txt

# Complete the merge
git commit
```

Resolving conflicts

Conflicts are marked in files like this:

```
<<<<<<< HEAD
Current branch content
=======
Incoming branch content
>>>>>>> feature-branch
```

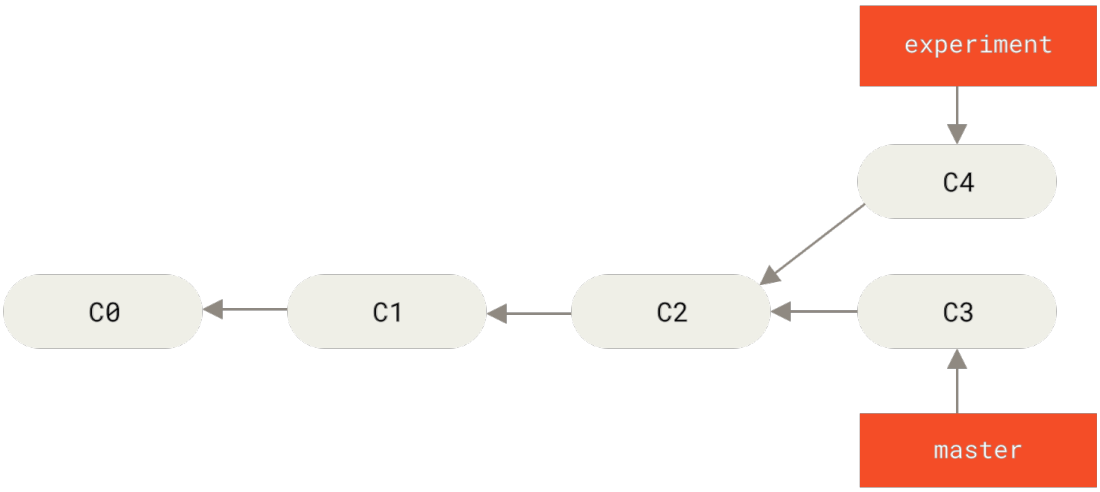
Resolving conflicts need understanding the type of modifications that were made in both branches.

For simple conflicts (e.g. a configuration option) you can usually take one of the two branches. In complex modifications it requires to actually work on the merging.

Visual Studio Code and main editors can visually help you.

Rebasing

Let's say we have two branches we want to merge:

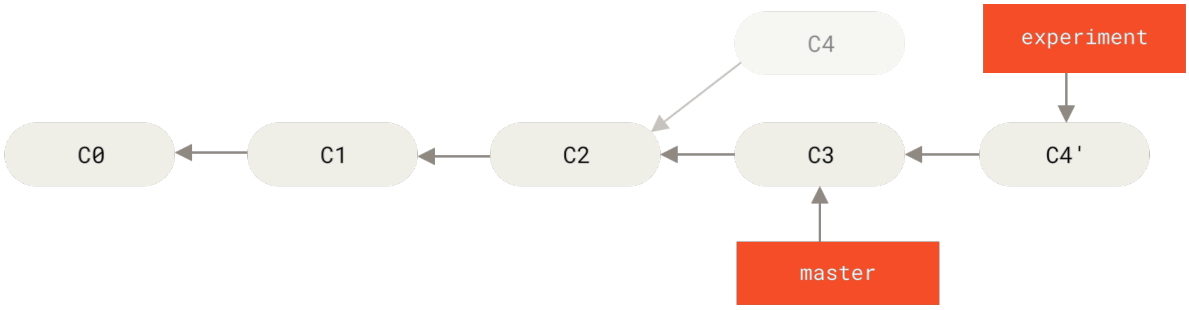


Rebasing

However, there is another way: you can take the patch of the change that was introduced in C4 and reapply it on top of C3.
In Git, this is called **rebasing:**

git checkout experiment

git rebase master

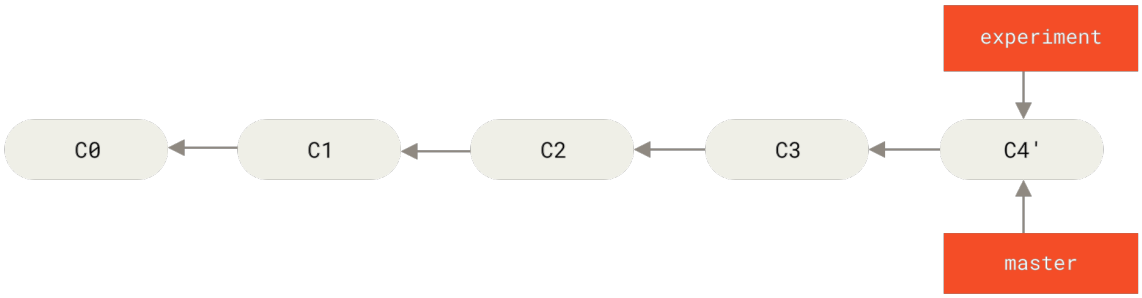


Rebasing

To complete it, you have to fast-forward master and remove experiment:

git checkout master

git merge experiment



Rebasing

Rebasing is an alternative to merging that rewrites history by applying commits on top of another base:

```
# Switch to the feature branch
git checkout feature-branch

# Rebase onto main
git rebase main
```

Benefits:

- Creates a cleaner, linear history
- Avoids unnecessary merge commits

Drawbacks:

- Rewrites commit history (don't rebase shared branches)
- Can make conflict resolution more complex

Rebasing

When to use rebase ✅

- Before making a merge: to keep the history clean.
- To update a branch with respect to main without creating merge commit.
- To modify past commits (with rebase -I, interactive mode).

When to avoid it ❌

- On a shared branch, because it rewrites the history and can create conflicts that are difficult to manage.
- If you have already pushed the commits and others are working on it.

If you have already pushed and done a rebase, you can use:

git push --force

But be careful: this rewrites the story and can create problems for collaborators

Stashing

Git stash temporarily saves changes you're not ready to commit.
Useful when you need to switch branches with uncommitted changes.

```
# Save changes and reset working directory
git stash

# List saved stashes
git stash list

# Apply the most recent stash
git stash apply

# Apply a specific stash
git stash apply stash@{2}

# Apply and remove the most recent stash
git stash pop

# Drop a stash
git stash drop stash@{1}

# Clear all stashes
git stash clear
```


Exercise

Exercise 3: branching and merging

- Create a feature branch
- Make several commits
- Switch back to main and make a different commit
- Merge your feature branch
- Resolve any conflicts

Exercise 4: stashing changes

- Create a feature branch
- Make several commits
- Create another feature branch
- Make changes
- Stash the changes, switch back to main and apply the stash

Remote Repositories

Remotes are repositories hosted elsewhere, usually on a server:

```
# List remote repositories
git remote -v

# Add a remote
git remote add origin https://github.com/username/repo.git

# Rename a remote
git remote rename origin upstream

# Remove a remote
git remote remove upstream
```

The default remote is typically called "origin".

Fetching and pulling

Fetching and pulling retrieve changes from remote repositories:

- **Fetch**: downloads changes but doesn't integrate them

```
git fetch origin
git fetch --all  # Fetch from all remotes
```

- **Pull:** fetches and merges changes in one step

```
git pull origin main
git pull --rebase  # Rebase instead of merge
```

It's often safer to fetch and then merge/rebase explicitly to better understand what's happening.

Pushing

Pushing sends your local commits to a remote repository:

```
# Push to the default remote branch
git push

# Push to a specific remote and branch
git push origin feature-branch

# Set upstream and push
git push -u origin feature-branch

# Force push (use with caution!)
git push --force
```

Force pushing overwrites the remote history and should be used carefully, especially on shared branches.

Tags

Tags mark specific points in Git history, typically for releases:

```
# List tags
git tag

# Create a lightweight tag
git tag v1.0.0

# Create an annotated tag with a message
git tag -a v1.0.0 -m "Release version 1.0.0"

# Tag a specific commit
git tag -a v0.9.0 -m "Beta release" 9fceb02

# Push tags to remote
git push origin v1.0.0      # Push a specific tag
git push origin --tags      # Push all tags
```

Annotated tags contain extra metadata like the tagger name, email, and date.

Advanced Git Features

Some advanced Git features worth exploring:

- **Git hooks**: Scripts that run at specific points in Git's workflow
- **Submodules**: Include other Git repositories within yours
- **Git LFS**: Handle large files efficiently
- **Aliases**: Create shortcuts for common commands
- **Rerere**: Automatically resolve repeated merge conflicts
- **Reflog**: Recover accidentally deleted commits
- **Bisect**: Binary search to find which commit introduced a bug

Best practices

Some best practices for using Git effectively:

- Commit early and often
- Keep commits focused on a single task
- Write meaningful commit messages
- Use branches for features and bug fixes
- Pull and push regularly
- Run tests before pushing
- Use .gitignore for build artifacts and dependencies
- Keep your repositories clean **(no large binary files)**
- Regularly backup your repositories
- Stay current with Git updates

Troubleshooting

I did something terribly wrong, can I go back in time?

```
git reflog
# you will see a list of every thing you've
# done in git, across all branches!
# each one has an index HEAD@{index}
# find the one before you broke everything

git reset HEAD@{index}
# magic time machine
```

You can use this to get files you accidentally deleted, or just to remove some actions you tried that broke the repo, or to recover after a bad merge, or just to go back to a time when things actually worked.

Troubleshooting

I just committed and immediately realised I need to make one small change!

```
# make your change
git add . # or add individual files
git commit --amend --no-edit
# now your last commit contains that change!
# WARNING: never amend public commits
```

You could also make the change as a new commit and then do rebase -i in order to squash them both together, but this is times faster.

Warning: You should never amend commits that have been pushed up to a public/shared branch! Only amend commits that only exist in your local copy or you're gonna have a bad time.

Troubleshooting

I need to change the message on my last commit!

```
git commit --amend
# follow prompts to change the commit message
```


Troubleshooting

I accidentally committed something to master that should have been on a brand new branch!

```
# create a new branch from the current state of master
git branch some-new-branch-name
# remove the last commit from the master branch
git reset HEAD~ --hard
git checkout some-new-branch-name
# your commit lives in this branch now :)
```

This doesn't work if you've already pushed the commit to a public/shared branch, and if you tried other things first, you might need to git reset HEAD@{number-of-commits-back} instead of HEAD~.

Troubleshooting

I accidentally committed to the wrong branch!

```
# undo the last commit, but leave the changes available
git reset HEAD~ --soft
git stash
# move to the correct branch
git checkout name-of-the-correct-branch
git stash pop
git add . # or add individual files
git commit -m "your message here";
# now your changes are on the correct branch
```

This doesn't work if you've already pushed the commit to a public/shared branch, and if you tried other things first, you might need to git reset HEAD@{number-of-commits-back} instead of HEAD~.

Troubleshooting

I tried to run a diff but nothing happened?!

If you know that you made changes to files, but diff is empty, you probably add-ed your files to staging and you need to use a special flag.

git diff --staged

Troubleshooting

I need to undo a commit from like 5 commits ago!

you don't have to track down and copy-paste the old file contents into the existing file in order to undo changes

If you committed a bug, you can undo the commit all in one go with revert.

```
# find the commit you need to undo
git log
# use the arrow keys to scroll up and down in history
# once you've found your commit, save the hash
git revert [saved hash]
# git will create a new commit that undoes that commit
# follow prompts to edit the commit message
# or just save and commit
```


Troubleshooting

I need to undo my changes to a file!

```
# find a hash for a commit before the file was changed
git log
# use the arrow keys to scroll up and down in history
# once you've found your commit, save the hash
git checkout [saved hash] -- path/to/file
# the old version of the file will be in your index
git commit -m "Wow, you don't have to copy-paste to undo"
```

Troubleshooting

All is lost, I give up fixing.

```
cd ..
sudo rm -r git-repo-dir
git clone https://some.github.url/git-repo-dir.git
cd git-repo-dir
```

In alternative, avoids redownloading the entire repository:

```
# get the lastest state of origin
git fetch origin
git checkout master
git reset --hard origin/master
# delete untracked files and directories
git clean -d --force
# repeat checkout/reset/clean for each borked branch
```


GitHub 

GitHub is a web-based hosting service for Git repositories that adds collaboration features:

- Repository hosting
- Pull requests for code review
- Issue tracking
- Project management
- Actions for CI/CD
- Wikis and documentation
- Social coding features

It's currently the largest host of source code in the world and a central hub for open-source projects.

GitHub repositories

On GitHub, you can:

- **Create a new repository**:

1. Click "New repository" on your dashboard
2. Enter a repository name
3. Add an optional description
4. Choose public or private
5. Initialize with a README if desired
6. Add a license and .gitignore if needed

- **Fork an existing repository**:

1. Navigate to the repository
2. Click "Fork" in the top-right corner
3. Choose where to fork it

Connecting local to remote

Connect your local repository to GitHub:

- **For a new repository**:

```
# Initialize local repository
git init
git add .
git commit -m "Initial commit"

# Connect to GitHub
git remote add origin https://github.com/username/repo.git
git push -u origin main
```

Connecting local to remote

- **For an existing repository**:

```
# Clone the repository
git clone https://github.com/username/repo.git
cd repo

# Make changes
git add .
git commit -m "Make changes"
git push
```


Protocols

On GitHub, you can push to two types of URL addresses:

An HTTPS URL like *https://github.com/user/repo.git*

An SSH URL, like *git@github.com:user/repo.git*

These can be your *origin* repos.

Cloning with HTTPS://

The https:// clone URLs are available on all repositories, regardless of visibility.

https:// clone URLs work even if you are behind a firewall or proxy.

When you *git clone*, *git fetch*, *git pull*, or *git push* to a private remote repository using HTTPS URLs on the command line, Git will ask for your GitHub username and password.

When Git prompts you for your password, enter your **personal access token** (you can create one from GitHub Settings)**.**

Cloning with SSH://

SSH URLs provide access to a Git repository via SSH, a secure protocol.

To use these URLs, you must generate an SSH keypair on your computer and add the public key to your account on GitHub.

1. Generate an SSH key:
   ```
   ssh-keygen -t ed25519 -C "your_email@example.com"
   ```
2. Add the key to the ssh-agent:
   ```
   ssh-add ~/.ssh/id_ed25519
   ```
3. Add the public key to your GitHub account
4. Use SSH URLs for your remotes:
   ```
   git@github.com:username/repo.git
   ```

GitHub Workflow

A typical GitHub collaboration workflow:

1. **Fork** the repository to your account
2. **Clone** your fork locally
3. Create a **branch** for your feature/fix
4. Make changes and **commit** them
5. **Push** your branch to your fork
6. Create a **pull request** to the original repository
7. Respond to **code review** feedback
8. Maintainer **merges** your pull request
9. **Sync** your fork with the upstream repository

Pull requests

Pull Requests (PRs) are the heart of GitHub collaboration:

1. Push your branch to GitHub:
   ```
   git push -u origin feature-branch
   ```
2. Go to your repository on GitHub
3. Click "Compare & pull request"
4. Add a title and description
5. Select the base branch (where changes should go)
6. Create the pull request

A good PR includes:

- Clear title and description
- Specific changes (not too large)
- Tests if applicable
- Documentation updates
- Reference to related issues

Code reviews

Code reviews are essential for maintaining code quality.

They are methodical assessments of code designed to identify bugs, increase code quality, and help developers learn the source code.

After a software developer has completed coding, a code review is an important step in the software development process to get a second opinion on the solution and implementation before it's merged into an upstream branch like a feature branch or the main branch.

Code reviews

As a reviewer:

- Be respectful and constructive
- Focus on the code, not the person
- Explain why changes are needed
- Suggest alternatives
- Approve when satisfied

As a code author:

- Be receptive to feedback
- Explain your reasoning
- Make requested changes or explain why not
- Push new commits to the same branch

Issues and Project management

GitHub helps in tracking bugs, features and tasks. For each of these, an issue can be created on the platform and handled by maintainers.

Anyone can join the discussion.

When creating issues, be sure to use:

- Clear title summarizing the problem/feature
- Detailed description with steps to reproduce
- Labels for categorization
- Assignees for responsibility
- Milestones for grouping

They are also organized into columns (in fashion similar to other systems):

To Do - In Progress - Review - Done

GitHub Actions

GitHub Actions enables CI/CD workflows directly in your repository. In practices, it is possible to create several “actions” or jobs, that on some trigger, execute steps like run unit tests.

1. Create a *.github/workflows* directory
2. Add YAML workflow files
3. Define triggers (e.g., push, pull request)
4. Specify jobs and steps
5. Use predefined or custom actions

GitHub Pages

GitHub Pages hosts websites directly from your repository:

1. Create a repository named *username.github.io*
2. Add HTML/CSS/JS files or use a static site generator
3. Push to GitHub
4. Your site will be available at *https://username.github.io*

You can also create project sites by enabling Pages in the repository settings.

Exercises

Exercise 5: GitHub collaboration - in groups of two or three

Each student creates a repository

For each repository:

Clone your forks

Create a branch and make changes

Push your branch

Create a pull request

The repo owner reviews and merges the changes

Exercise 6: GitHub collaboration - in groups of two or three

Each student create an issue on the main repo

Proceed like in exercise 5 and link the pull request to the issue

The repo owner reviews, merges the changes and close the issue